

Devoir 1 - Méthodes Statistiques

“XXX”

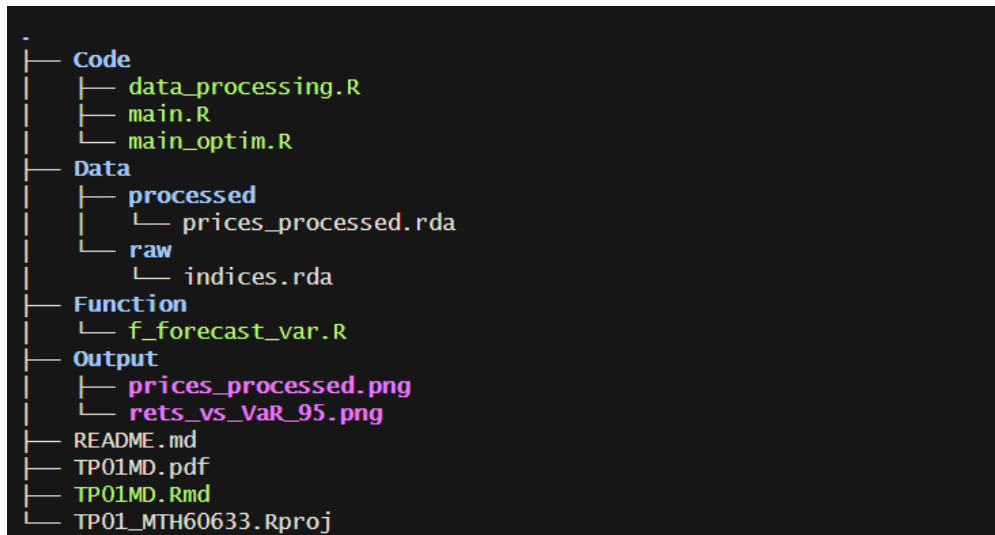
Hiver 2026

```
#install.packages("fs") #install.packages("here") #install.packages("zoo") #install.packages("xts")
library(fs)
library(here)
library(zoo)
library(xts)
library(PerformanceAnalytics)
source(here("Code", "data_processing.R"), echo = FALSE)
source(here("Function", "f_forecast_var.R"), echo = FALSE)
source(here("Function", "f_var_roll_wndw.R"), echo = FALSE)
source(here("Code", "main.R"), echo = FALSE)
knitr::read_chunk(here("Function", "f_forecast_var.R"))
knitr::read_chunk(here("Function", "f_var_roll_wndw.R"))
knitr::read_chunk(here("Code", "main.R"))
```

1. Structure du projet

Afin d’assurer la reproductibilité et une organisation claire du travail, le projet a été structuré en plusieurs dossiers fonctionnels. La structure comprend quatre dossiers principaux. Le dossier Code contient les scripts exécutables du projet, notamment le script de traitement des données ainsi que le script principal (main) permettant de reproduire l’ensemble des résultats demandés dans le TP. Le dossier Data regroupe les données utilisées dans le projet. Il est divisé en deux sous-dossiers : raw, qui contient les données brutes originales, et clean, qui contient les données nettoyées et prêtes à être utilisées pour l’analyse. Le dossier Function contient les fichiers .R regroupant les fonctions utilisées dans le projet, par exemple les fonctions d’estimation GARCH, de calcul de la VaR et les fonctions utilitaires. Enfin, le dossier Output contient tous les résultats générés automatiquement par le code. Cette organisation permet une exécution automatisée et indépendante de la plateforme tout en facilitant la maintenance et la lisibilité du projet.

Illustration de la structure



Travail collaboratif et gestion de version

Afin de faciliter le travail en équipe et d'assurer le suivi des modifications, le projet est hébergé sur GitHub :

https://github.com/Ulrizpascuit/TP01_MTH60633.git

Sur ce répertoire, une branche distincte a été créée pour chaque coéquipier. Cette approche permet de travailler simultanément sans conflits et de récupérer facilement les mises à jour des autres membres.

2. Données

2.1 Filtrage des données

Premièrement, il a fallu charger le fichier `indices.rda` qui contient un objet, nommé `prices`, de type `xts`. Cet objet contient les prix journaliers du SP500 et du FTSE100 de l'année 1990 à 2013. Le code `data_processing.R` permet de garder seulement les valeurs depuis janvier 2005. Ces valeurs sont sauvegardées dans un autre fichier nommé `prices_processed.rda`. C'est ce fichier qui sera chargé pour le travail. La figure 1 présente les données filtrées qui seront utilisées pour le reste du TP.

2.2 Calcul des log-rendements

Puisqu'on s'intéresse à faire de la prévision de risque sur les log-rendements des indices, il faut les calculer. La formule utilisée pour calculer les log-rendements est la suivante:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

En pratique, on utilise une fonction qui le calcul directement. La figure 2 présente les log-rendements des deux indices.

On peut voir rapidement un comportement typique de log-rendement de séries financière: centrée approximativement en 0 et avec une variance qui semble dépendre du temps. L'intuition de coller un modèle GARCH à ces données est donc acceptable. La suite de ce travail reposera donc sur ces deux séries de données.

3. Explications des fonctions

Afin d'arriver à calculer la VaR next-step ahead du modèle GARCH(1,1) collé à nos données, le fichier de fonction nommé `f_forecast_var.R` a été utilisé. Ce fichier contient 3 fonctions dont `f_forecast_var` qui calcul directement la VaR à un pas de temps futur. Toutes les fonctions du fichiers ont été corrigées afin qu'elles fonctionnent comme elles se doivent.

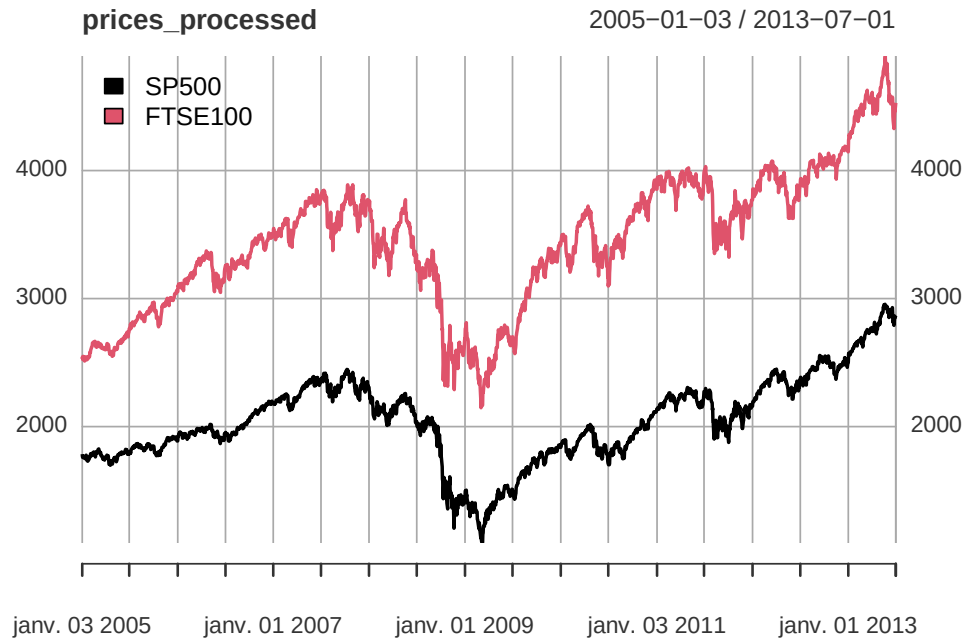


Figure 1: Valeur des indices (SP500 et FTSE100) depuis janvier 2005

3.1 Fonction `f_ht`

Premièrement, on a la fonction `f_ht` qui calcule la variance inconditionnelle du modèle à chaque pas de temps. Cette fonction nécessite en entrée les paramètres du modèle GARCH(1,1) et le vecteur des log Return des indices. La correction de cette fonction était d'ajouter le calcul de la variance conditionnelle. Au premier pas de temps, la variance conditionnelle est approximée par la variance inconditionnelle du modèle:

$$\sigma_1^2 = \frac{\omega}{1 - \alpha - \beta}$$

Par la suite, la variance conditionnelle est calculée par la formule du modèle GARCH(1,1):

$$\sigma_t^2 = \omega + \alpha y_{t-1}^2 + \beta \sigma_{t-1}^2$$

Afin de calculer la variance à chaque pas de temps, une boucle sur tous les pas de temps a été utilisée. Le code suivant montre la fonction corrigée.

```
f_ht <- function(theta, y) {
  ### Function which computes the vector of conditional variance
  # INPUTS
  # x0 : [vector] (3 x 1)
  # y   : [vector] (T x 1) log-returns
  # OUTPUTS
  # sig2 : [vector] (T+1 x 1) conditional variances

  # Extract the parameters
  a0 <- theta[1]
  a1 <- theta[2]
  b1 <- theta[3]
```

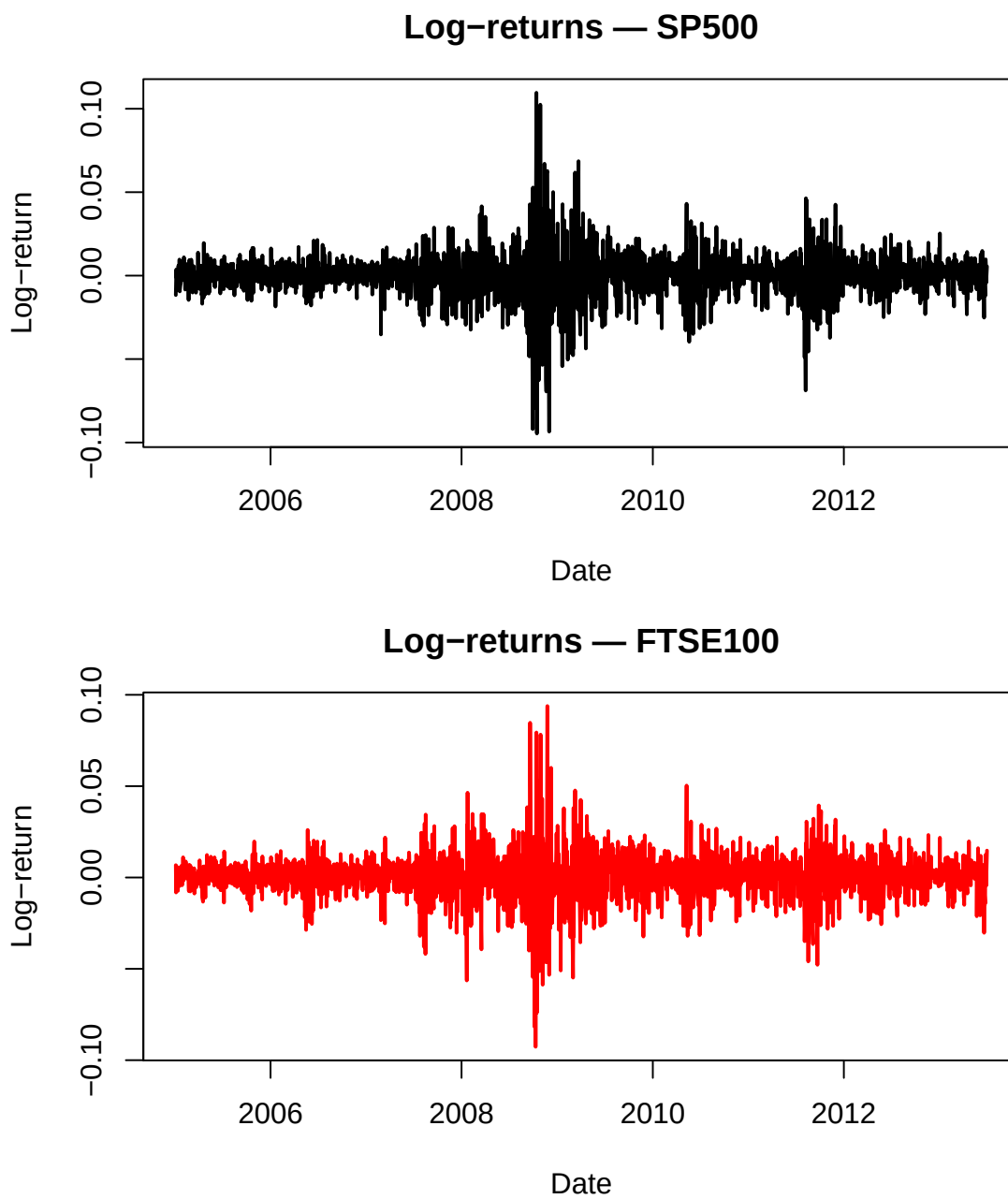


Figure 2: Valeur des indices (SP500 et FTSE100) depuis janvier 2005

```

T <- length(y)

# Initialize the conditional variances
sig2 <- rep(NA, T + 1)

# Start with unconditional variances
sig2[1] <- a0 / (1 - a1 - b1)

# Compute conditional variance at each step
### !!! FIXME !!!
for (i in 2:(T+1)) {
  sig2[i] <- a0 + a1 * y[i-1]^2 + b1 * sig2[i-1]
}
sig2
}

```

3.2 Fonction f_nll

Deuxièmement, la fonction `f_nll` est utilisée pour calculer la log-vraisemblance d'un modèle. Cette fonction est très importante, car c'est minimisant la valeur retournée par celle-ci qu'on arrive à obtenir des paramètres optimaux pour le modèle. Ici, on suppose que les données suivent une loi normale centrée en 0 avec une certaine variance. La log-vraisemblance des données est donc calculée en sommant la densité d'une loi normale centrée en 0 et avec une variance pour chaque pas de temps qui correspond à la variance conditionnelle calculée par la fonction `f_ht`. On a donc $y_t \mid \mathcal{F}_{t-1} \sim \mathcal{N}(0, \sigma_t^2)$ où σ_t^2 est la variance conditionnelle produite par la fonction `f_ht`. La densité conditionnelle associée est donc :

$$f(y_t \mid \mathcal{F}_{t-1}) = \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\left(-\frac{y_t^2}{2\sigma_t^2}\right).$$

La log-vraisemblance sur $t = 1, \dots, T$ s'écrit alors :

$$\ell(\theta) = \sum_{t=1}^T \left[-\frac{1}{2} \log(2\pi) - \frac{1}{2} \log(\sigma_t^2) - \frac{1}{2} \frac{y_t^2}{\sigma_t^2} \right]$$

Le code suivant montre la fonction corrigée.

```

f_nll <- function(theta, y) {
  ### Fonction which computes the negative log likelihood value
  ### of a GARCH model with Normal errors
  # INPUTS
  # theta : [vector] of parameters
  # y      : [vector] (T x 1) of observations
  # OUTPUTS
  # nll    : [scalar] negative log likelihood value

  T <- length(y)

  # Compute the conditional variance of a GARCH(1,1) model
  sig2 <- f_ht(theta, y)

  # Consider the T values
  sig2 <- sig2[1:T]

  # Compute the loglikelihood

```

```
## !!! FIXME !!!
ll <- sum(-1/2 * (log(2*pi) + log(sig2) + y**2/sig2))

# Output the negative value
nll <- -ll

nll
}
```

3.3 Fonction `f_forecast_var`

Troisièmement, la fonction `f_forecast_var` est utilisée afin de calculer la prévision de la Value-at-Risk (VaR) à un pas de temps futur à partir d'un modèle GARCH(1,1) avec erreurs normales. Cette fonction nécessite en entrée le vecteur des log-returns des indices ainsi qu'un niveau de risque `level` (par exemple 0.95 pour une VaR au niveau de 95%). La première étape de cette fonction consiste à estimer les paramètres du modèle GARCH(1,1) par maximum de vraisemblance. Pour ce faire, la fonction `f_nll` définie précédemment est minimisée numériquement. La première correction apportée était la définition des contraintes du problème d'optimisation. Une borne inférieure sur les paramètres doit être définie afin de garantir que ceux-ci demeurent positifs et la condition de stationnarité du modèle GARCH(1,1) doit également être imposée. On se retrouve donc avec les contraintes suivantes:

$$\omega > 0, \quad \alpha > 0, \quad \beta > 0, \quad \alpha + \beta < 1$$

Ces contraintes sont intégrées dans la fonction d'optimisation `constrOptim` (discuté plus loin) sous forme de contraintes linéaires. On doit donc les écrire sous la forme $A\theta \geq b$ où θ est le vecteur des paramètres (ω, α, β) . Sous forme matricielle, on a donc:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & -1 \end{pmatrix}, \quad \theta = \begin{pmatrix} \omega \\ \alpha \\ \beta \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & -1 \end{pmatrix} \begin{pmatrix} \omega \\ \alpha \\ \beta \end{pmatrix} \geq \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

La deuxième correction a été de fournir le code pour l'optimisation du modèle. Ici, la méthode utilisée pour trouver les paramètres optimaux du modèle est de minimiser le négatif du log-vraisemblance. Pour ce faire, nous utilisons la fonction `constrOptim`, pour minimiser la valeur de sortie de la fonction `f_nll` présentée plus tôt. Les contraintes sont directement introduites dans les entrées de l'optimiseur et le vecteur y des log-rendements est ajouté à la fin des entrées de l'optimiseur afin qu'il puisse l'utiliser comme entrée dans l'appel de `f_nll` (car `f_nll` n'a pas que θ comme entrée).

La troisième correction a été de calculer la VaR d'un pas de temps futur selon le niveau de tolérance `level`. Étant donnée qu'on suppose que y_t suit une loi normale, alors on peut obtenir la VaR en calculant le quantile d'une loi normale de variance σ_{t+1}^2 (variance conditionnelle estimée par le modèle) associée au `level` voulu.

$$VaR_{t+1} = q_{1-\text{level}} \sqrt{\sigma_{t+1}^2}$$

Le code suivant montre la fonction corrigée.

```
f_forecast_var <- function(y, level) {
  ### Compute the VaR forecast of a GARCH(1,1) model with Normal errors at the desired risk level
  # INPUTS
  # y      : [vector] (T x 1) of observations (log-returns)
  # level  : [scalar] risk level (e.g. 0.95 for a VaR at the 95# risk level)
  # OUTPUTS
  # VaR    : [scalar] VaR forecast
  # sig2    : [vector] (T+1 x 1) conditional variances
  # theta   : [vector] GARCH parameters
```

```

# NOTE
# o the estimation is done by maximum likelihood
eps <- 1e-8

y <- as.numeric(y)
y <- y[is.finite(y)]

# Fit a GARCH(1,1) model with Normal errors
# Starting values and bounds
theta0 <- c(0.1 * var(y), 0.1, 0.8)
LB      <- ## !!! FIXME !!! ----> g  r   dans les contraintes lin  aires A*theta >= b
# Stationarity condition
A       <- matrix(c(1,0,0,0,1,0,0,0,1,0,-1,-1), nrow = 4, byrow = TRUE)
b       <- c(eps,eps,eps,-(1-eps))

# Run the optimization
## !!! FIXME !!!
fit <- constrOptim(theta = theta0,
                  f_nll,
                  ui = A,
                  ci = b,
                  grad = NULL,
                  y = y,
                  control = list(reltol = 1e-10))

theta <- fit$par
# Recompute the conditional variance
sig2 <- f_ht(theta, y)

# Compute the next-day ahead VaR for the Normal model
## !!! FIXME !!!
VaR <- qnorm(1-level) * sqrt(sig2[length(sig2)])

out <- list(VaR_Forecast = VaR,
            ConditionalVariances = sig2,
            GARCH_param = theta)

out
}

```

4. Estimation statique de la VaR

  partir des fonctions corrig  es de la section 3 et en fixant un niveau de risque, il est maintenant possible d'utiliser la fonction `f_forecast_var` pour faire une premi  re estimation statique de la VaR des deux indices. Pour se faire, on isole une fen  tre d'estimations qui correspond aux 1000 premiers jours de notre  chantillon pour chacun des indices. On applique ensuite la fonction `f_forecast_var` pour mod  liser leur volatilit   respective selon un mod  le GARCH(1,1)   un seuil de confiance de 95%. On extrait les r  sultats   partir de l'objet VaR de la liste de sortie fournie par la fonction, ce qui nous permet d'obtenir l'estimation de la perte journali  re maximale anticip  e par notre mod  le pour chaque indice.

Table 1: Estimation statique de la VaR   95%   l'horizon T+1

Indice	VaR_95
SP500	-0.06943

Indice	VaR_95
FTSE100	-0.06237

Au niveau de risque de 95%, la VaR estimée à l'horizon T+1 est négative puisqu'elle représente une perte (quantile gauche) des rendements. L'indice le plus risqué est celui dont la VaR est la plus faible (la plus négative). On peut voir à partir du tableau 1 que c'est le SP500 qui a la VaR la plus importantes (environ -6.9%) comparativement au FTSE100 (environ -6.2%). Étant donnée qu'on dit qu'on actif avec une plus grande variance est considéré comme plus risqué, alors on peut dire que qu'à l'horizon T+1, le SP500 est les plus risqué des deux indices.

5. Backtesting

5.1 Méthodologie et résultats

Afin d'évaluer la capacité prédictive du modèle GARCH(1,1), une procédure de backtesting est mise en place. L'objectif est de simuler une situation réelle dans laquelle la VaR est estimée uniquement à partir de l'information disponible au moment de la prévision. Pour ce faire, une fenêtre glissante de taille T=1000 observations est utilisée. À chaque itération, le modèle GARCH(1,1) est estimé à partir des 1000 rendements logarithmiques les plus récents disponibles. Plus précisément, pour chaque $t = 1001, \dots, 2000$, on considère la fenêtre de données:

$$y_{\text{window}} = \{y_{t-1000}, \dots, y_{t-1}\}$$

On a donc 1000 $y_{t-\text{window}}$ pour obtenir 1000 ensembles de paramètres optimaux pour ensuite calculer 1000 VaR à des pas de temps consécutifs. De manière plus formelles, on calcule:

$$VaR_t = q_{0.05} \sqrt{\sigma_t^2}, \quad \sigma_t^2 \text{ obtenu du modèle estimé par } y_{\text{window}} = \{y_{t-1000}, \dots, y_{t-1}\} \text{ avec } t = 1001, \dots, 2000$$

L'implantation de cette méthode de backtesting a été fait à l'aide de deux boucle **for** imbriquées: la première sert à passer tous les indices dont on souhaite calculer les 1000 VaR et la deuxième sert à passer les 1000 VaR à calculer. Une simple adaptation de la variable résultats d'une variable de type **double** à une variable de type **matrix** a été nécessaire comme changement pour bien afficher les résultats par la suite. Suite à des tests préliminaires, nous avons remarqué qu'il fallait plus d'une minutes pour calculer toutes les VaR. Afin d'accélérer la reproduction de résultats, l'objet contenant toutes les VaR a été sauvegardé dans un fichier .rda nommé **VaR_roll_xts.rda**. Ainsi, une condition a été ajouté dans le code afin de vérifier si le fichier existait déjà pour ne pas à recalculer à chaque fois. Le code qui suit présente la manière de vérifier:

```
file_var <- here("Output", "VaR_roll_xts.rda")
if (file.exists(file_var)) {
  load(file_var)
  valid_object <- exists("VaR_roll_xts") &&
    nrow(VaR_roll_xts) > 0 &&
    all(colnames(VaR_roll_xts) == colnames(logRets))
  if (!valid_object) {
    message("Fichier invalide. Recalcul de la VaR rolling.")
    VaR_roll_xts <- f_var_roll_wndw(logRets, window, h, level)
    save(VaR_roll_xts, file = file_var)
  }
} else {
  message("Fichier inexistant. Calcul de la VaR rolling.")
  VaR_roll_xts <- f_var_roll_wndw(logRets, window, h, level)
  save(VaR_roll_xts, file = file_var)
}
```

Ici, on vérifie si le fichier.rda contient un objet dont les noms des colonnes sont les mêmes que les noms des colonnes des données que nous avons chargées. On regarde aussi si la longueur de l'objet est non nulle. Ces conditions ne sont pas très robustes, mais plus que suffisantes dans le cadre de ce travail.

De plus, puisque l'ajout des conditions présentées précédemment, le code qui calcul toutes les VaR se retrouve dupliqués plusieurs fois. Ainsinous avons fait une fonction `f_var_roll_wndw` dans le fichier `f_var_roll_wndw.R` qui retourne directement toutes les VaR à calculer. La fonction est la suivante:

```
f_var_roll_wndw <- function(logRets, window = 1000, h = 1000, level = 0.95) {  
  
  n_indices <- ncol(logRets)  
  
  VaR_roll <- matrix(NA_real_, nrow = h, ncol = n_indices)  
  
  for (j in 1:n_indices) {  
    for (k in 1:h) {  
      y_window <- logRets[k:(window + k - 1), j]  
      out <- f_forecast_var(y_window, level)  
      VaR_roll[k, j] <- as.numeric(tail(out$VaR_Forecast, 1))  
    }  
  }  
  
  colnames(VaR_roll) <- colnames(logRets)  
  
  dates_var <- index(logRets)[(window + 1):(window + h)]  
  
  VaR_roll_xts <- xts(VaR_roll, order.by = dates_var)  
  return(VaR_roll_xts)  
}
```

5.2 Vérification et analyse des résultats